

PROPOSAL: GitGuard – The Autonomous Quality Engine

1. Executive Summary

The **GitGuard** initiative is a software testing competition designed to challenge participants to build a fully autonomous AI Testing Agent. Unlike traditional automation, which is reactive and brittle, GitGuard leverages Generative AI to "read" source code directly from GitHub, generate a comprehensive testing suite, and maintain it through self-healing mechanisms. This project aims to eliminate the manual overhead of test maintenance and data preparation.

2. Problem Statement

Modern software development moves at a speed that manual and even traditional automated testing cannot match. Key pain points include:

- **Maintenance Debt:** Automated scripts break every time the UI changes.
 - **Data Bottlenecks:** Creating realistic test data is time-consuming and carries privacy risks.
 - **Delayed Analysis:** When a test fails, developers spend hours finding the specific code change that caused it.
-

3. The Solution: Core Features

The competition requires participants to develop a system that addresses the following modules:

A. Intelligent Generation

- **Auto-Case Discovery:** The AI analyzes the GitHub repository to identify user flows and business logic, automatically generating text-based test cases.

- **Script Synthesis:** The AI converts these cases into executable scripts (e.g., Playwright, Selenium, or Cypress) without human coding.

B. Dynamic Environment & Data

- **Synthetic Data Generation:** Instead of using real user data, the AI generates high-fidelity, privacy-compliant synthetic data to populate forms and databases during execution.
- **Smart Scheduling:** The system prioritizes tests based on the "Risk Score" of a GitHub commit—testing the most changed or vulnerable code first.

C. Self-Healing & Root Cause Analysis (RCA)

- **Autonomous Healing:** If a button's ID or a page's structure changes, the AI detects the shift and automatically updates the test script to prevent "flaky" failures.
- **Enhanced Defect Reporting:** When a failure occurs, the AI provides a report linking the crash directly to the specific line of code in the GitHub commit that triggered it.

4. Technical Architecture (Proposed)

The competition entries should ideally follow this flow:

1. **Ingestion:** Webhook triggers on a GitHub Push/Pull Request.
2. **Analysis:** LLM-based parsing of the updated code files.
3. **Synthesis:** Generation of .spec.js or .py test files.
4. **Execution:** Run in a headless containerized environment.
5. **Healing:** Re-scan DOM on failure to suggest script fixes.

5. Judging Criteria & Scoring

Participants will be scored based on the following metrics:

Criteria	Description	Weight
Generation Accuracy	How well the AI identifies edge cases from the GitHub code.	25%
Self-Healing Efficacy	Ability to recover from UI changes without manual intervention.	25%

Criteria	Description	Weight
Data Realism	The quality and variety of the synthetic test data generated.	15%
RCA Depth	The precision of the AI in identifying the "Root Cause" in the source code.	20%
Innovation	Unique features (e.g., Predictive bug detection based on Git history).	15%
Export to Sheets		

6. Innovation Highlight: "Predictive Regression"

To push the boundaries of S/W testing, GitGuard introduces **Predictive Regression**. By analyzing the history of a GitHub repository, the AI identifies "Hotspots"—files that have historically been associated with the most bugs—and automatically doubles the test density for those specific modules during a build.

Thank you for reading fellas 😊